```
        return $this->quoteStash[rand(0,2)];
    }
}
```

Now we have a class that implements an interface and fulfils the requirement of implementing the interface's `getQuote()` method. The class implementation must be compatible with the interface declaration: no changing return types or parameters is allowed, and visibility must remain public.

So far this looks like a lot of hassle, but it is useful because interfaces can be used as type declarations:

```
function useQuoteBot(QuoteBotInterface $bot): void
{
    print $bot->getQuote();
}
```

Now we have a function that relies on the interface of the object passed into it: the class is irrelevant so long as it implements `QuoteBotInterface`. Let's build a couple of classes that make use of it:

```
class LebowskiBot implements QuoteBotInterface
{
    // quoteStash array and getQuote() implementation
}

class FinkBot implements QuoteBotInterface
{
    // quoteStash array and getQuote() implementation
}

function useQuoteBot(QuoteBotInterface $bot): void
{
    print $bot->getQuote();
}

$lebowski = new LebowskiBot();
$fink = new FinkBot();

useQuoteBot($lebowski);
useQuoteBot($fink);
```